```python
import grpc
from decimal import Decimal, InvalidOperation
import banking_pb2
import banking_pb2_grpc

SERVER_ADDRESS = "127.0.0.1:50051"
RPC_TIMEOUT = 5
MIN_INT64 = -(2**63)
MAX_INT64 = 2**63 - 1

def rupees_to_paise(value):
    """Convert a rupee amount to integer paise."""
    try:
        amount = Decimal(value.strip())

        if not amount.is_finite():
            raise ValueError("Amount must be finite.")

        paise_decimal = amount * 100

        if paise_decimal != paise_decimal.to_integral_value():
            raise ValueError("Amount cannot have fractional paise.")

        if not MIN_INT64 <= paise_decimal <= MAX_INT64:
            raise ValueError("Amount is outside the supported range.")

        return int(paise_decimal)

    except (InvalidOperation, OverflowError):
        raise ValueError("Please enter a valid monetary amount.")
```

(grpc library)

message class

from python built in module

service class

generated message classes by banking proto

used for money calculations.

throwing exceptions.

Loopback TCP port

```
banking_pb2_grpc.AccountManagementServiceStub(channel)
Creates a client side stub, allowing to invoke method on server.
```

| File | Purpose |
|---|---|
| `banking_pb2.py` | Defines the request and response message types |
| `banking_pb2_grpc.py` | Defines client stubs and server registration functionality |

Deadline-exceed status.

String

since proto buf uses int64 we make sure a monetary value can be handle by client.
$(-2^{63}$ to $+2^{63}-1)$.

convert to a monetary value.

because we have.

```
Python
Decimal("Infinity")
Decimal("NaN")
```

convert.

check conversion. → rejects 10.505

check range

return integer.

generates a more user friendly error.

```python
def format_money(paise):
    """Convert integer paise to a readable rupee amount."""
    return f"Rs. {Decimal(paise) / 100:,.2f}"


def read_account_id(prompt):
    """Read and validate an account ID."""
    user_id = input(prompt).strip()

    if not user_id:
        raise ValueError("Account ID cannot be empty.")

    return user_id


def get_balance(account_stub):
    user_id = read_account_id("Enter account ID: ")
    response = account_stub.getBalance(
        banking_pb2.BalanceRequest(user_id=user_id),
        timeout=RPC_TIMEOUT
    )

    print(f"\nAccount ID: {response.user_id}")
    print(f"Current Balance: {format_money(response.balance_paise)}")
```

— upto to two decimal points (helper function).

— check for empty.

The `account_stub` parameter holds a client-side proxy object created from the generated gRPC code.

Actual gRPC call

— create a BalanceRequest message object

time out we defined earlier.

then account_stub invokes the procedure.

(it is a blocking synchronous call).

Print response.

```python
def update_balance(account_stub):
    user_id = read_account_id("Enter account ID: ")
    amount = input("Enter new balance (Rs.): ")

    new_balance_paise = rupees_to_paise(amount)

    if new_balance_paise < 0:
        raise ValueError("New balance cannot be negative.")
    response = account_stub.updateBalance(
        banking_pb2.UpdateBalanceRequest(
            user_id=user_id,
            new_balance_paise=new_balance_paise
        ),
        timeout=RPC_TIMEOUT
    )

    print(f"\n{response.message}")

    if response.success:
        print(
            f"Updated Balance: "
            f"{format_money(response.updated_balance_paise)}"
        )
    else:
        print("Balance update failed.")
```

← helper function. above.

✓ check for negative balance update.

actual gRPC call.

} create update request message.

1. Python constructs the `BalanceRequest` message.
2. The gRPC stub serializes the request using Protocol Buffers.
3. The request is transmitted over HTTP/2 to the server.
4. The server dispatches it to its `getBalance()` implementation.
5. The server returns a `BalanceResponse` message.
6. gRPC deserializes the response and returns it to your client.

```python
def transfer_money(transaction_stub):
    sender = read_account_id("Enter sender account ID: ")
    receiver = read_account_id("Enter receiver account ID: ")

    if sender == receiver:
        raise ValueError(
            "Sender and receiver accounts must be different."
        )

    amount = input("Enter transfer amount (Rs.): ")
    amount_paise = rupees_to_paise(amount)

    if amount_paise <= 0:
        raise ValueError(
            "Transfer amount must be greater than zero."
        )

    response = transaction_stub.initiateTransfer(
        banking_pb2.TransferRequest(
            from_user_id=sender,
            to_user_id=receiver,
            amount_paise=amount_paise
        ),
        timeout=RPC_TIMEOUT
    )

    print(f"\n{response.message}")

    if response.success:
        print(f"Transaction ID: {response.transaction_id}")
    else:
        print("Transfer failed.")
```

same account transfer.

take input.

check for negative.

create message and call gRPC.

```python
def transaction_history(transaction_stub):
    user_id = read_account_id("Enter account ID: ")

    response = transaction_stub.getTransactionHistory(
        banking_pb2.TransactionHistoryRequest(user_id=user_id),
        timeout=RPC_TIMEOUT
    )

    print(f"\nTransaction History for Account {user_id}")

    if not response.transactions:
        print("No transactions found.")
        return

    for index, transaction in enumerate(response.transactions, 1):
        print(f"\nTransaction {index}")
        print(f"ID: {transaction.transaction_id}")
        print(f"From: {transaction.from_user_id}")
        print(f"To: {transaction.to_user_id}")
        print(f"Amount: {format_money(transaction.amount_paise)}")
        print(f"Timestamp: {transaction.timestamp}")
```

as defined in repeated in proto buffer.

print each transaction.

```python
def handle_grpc_error(error):
    """Display useful information about RPC failures."""
    code = error.code()

    print(f"\ngRPC Error: {code.name}")
    print(f"Details: {error.details()}")

    if code == grpc.StatusCode.UNAVAILABLE:
        print("The banking server may be offline or unreachable.")

    elif code == grpc.StatusCode.DEADLINE_EXCEEDED:
        print("The request timed out.")
        print("The operation may still have completed on the server.")

    elif code == grpc.StatusCode.INVALID_ARGUMENT:
        print("The server rejected the request arguments.")

    elif code == grpc.StatusCode.NOT_FOUND:
        print("The requested resource was not found.")
```

function to gracefully handle errors.

**What is `error`?**

When an RPC fails, gRPC raises a `grpc.RpcError` exception.

This exception provides information about what went wrong.

`code = error.code()`

Retrieves the gRPC status code associated with the failure.

```python
def main():
    try:
        with grpc.insecure_channel(SERVER_ADDRESS) as channel:

            account_stub = (
                banking_pb2_grpc.AccountManagementServiceStub(channel)
            )

            transaction_stub = (
                banking_pb2_grpc.TransactionServiceStub(channel)
            )

            while True:
                print("\n========== BANKING SYSTEM ==========")
                print("1. Check Account Balance")
                print("2. Update Account Balance")
                print("3. Transfer Money")
                print("4. View Transaction History")
                print("5. Exit")
```

Creates the client-side proxy for `AccountManagementService`.

client side proxy for Transaction service

`with grpc.insecure_channel(SERVER_ADDRESS) as channel:`

Creates a gRPC communication channel.

The channel represents a connection-management abstraction used to communicate with the server.

The `with` statement ensures the channel is closed when the block exits.

A subtle but important detail: constructing a gRPC channel does not necessarily mean that a network connection has already been established. gRPC channels manage connections lazily and can reconnect as needed.

```
            try:
                choice = input(
                    "\nEnter your choice (1-5): "
                ).strip()

                if choice == "1":
                    get_balance(account_stub)

                elif choice == "2":
                    update_balance(account_stub)

                elif choice == "3":
                    transfer_money(transaction_stub)

                elif choice == "4":
                    transaction_history(transaction_stub)

                elif choice == "5":
                    print("Exiting Banking System.")
                    break

                else:
                    print("Invalid choice. Please select 1-5.")

            except ValueError as error:
                print(f"\nInput Error: {error}")

            except grpc.RpcError as error:
                handle_grpc_error(error)

    except (KeyboardInterrupt, EOFError):
        print("\nBanking client closed.")


if __name__ == "__main__":
    main()
```

Through it, your client can call:

Python

```
transaction_stub.initiateTransfer(...)
transaction_stub.getTransactionHistory(...)
```

Handles validation errors raised by your helper functions.

For example, if the user types:

Plain text

```
abc
```

when asked for an amount, the program displays an input error rather than terminating.

— for handling gRPC errors.

```
            try:
                choice = input(
                    "\nEnter your choice (1-5): "
                ).strip()

                if choice == "1":
                    get_balance(account_stub)

                elif choice == "2":
                    update_balance(account_stub)

                elif choice == "3":
                    transfer_money(transaction_stub)

                elif choice == "4":
                    transaction_history(transaction_stub)

                elif choice == "5":
                    print("Exiting Banking System.")
                    break

                else:
                    print("Invalid choice. Please select 1-5.")

            except ValueError as error:
                print(f"\nInput Error: {error}")

            except grpc.RpcError as error:
                handle_grpc_error(error)

    except (KeyboardInterrupt, EOFError):
        print("\nBanking client closed.")


if __name__ == "__main__":
    main()
```

```
except (KeyboardInterrupt, EOFError):
    print("\nBanking client closed.")
```

This outer handler catches two common situations.

`KeyboardInterrupt`

Usually raised when the user presses `Ctrl+C`.

`EOFError`

Can occur when `input()` reaches the end of the input stream, such as when the user presses `Ctrl+D` on a typical Linux terminal.

Instead of displaying a Python traceback, the client prints a short shutdown message.

Because the channel is managed by `with`, it is closed as the program exits the block.